

ES/AM 158 Lab 1 Upkie

Matthias Jammot

23 November 2025

Code: <https://github.com/MattJmt/2025-ES-AM-158-PSET.git>

1 Task 1: Finish Environment (10 points)

The **Upkie-Spine-Pendulum** environment exposes Upkie as a wheeled inverted pendulum, similar in spirit to the classic CartPole environment. The full dynamics are still simulated by the Spine physics backend, but the wrapper constrains the legs to remain straight and restricts actuation to the wheel velocities.

In the original Upkie Pendulum environment, the backend simulator returned a constant reward and only enforced a fixed 300-step time limit. As a result, an RL agent received no information about whether its actions were stabilizing the robot or leading toward failure. To make learning possible, I modified `upkie/envs/upkie_pendulum.py` by adding two essential components:

1.1 New Termination Conditions

Originally, the pendulum environment terminated only when `__detect_fall()` fired (large body pitch) or when the episode exceeded its fixed duration. I added additional physically meaningful failure conditions in the `step()` function to terminate episodes early whenever the pendulum state becomes unrecoverable. This reduces wasted rollout time and provides clearer learning signals.

- **Position Runaway:** to prevent the robot from drifting too far horizontally.

```
MAX_POSITION = 1.0 # m
if abs(p) > MAX_POSITION:
    terminated = True
```
- **Excessive angular velocity:** if the robot spins too fast, stabilization is no longer feasible.

```
MAX_THETA_DOT = 10.0 # rad/s
if abs(theta_dot) > MAX_THETA_DOT:
    terminated = True
```
- **Excessive horizontal velocity:** Stops episodes when the robot accelerates uncontrollably.

```
MAX_P_DOT = 3.0 # m/s
if abs(p_dot) > MAX_P_DOT:
    terminated = True
```

These conditions improved learning efficiency because episodes ended immediately upon unrecoverable failure, allowing the agent to gather more informative training data in the same amount of time.

1.2 Reward Shaping for Stabilization

The original environment returned a constant reward, which makes reinforcement learning impossible. I implemented a shaped reward using the pendulum-like state variables. The observation vector was:

$$o = [\theta, p, \dot{\theta}, \dot{p}]$$

and the new reward function was designed to encourage the robot to remain upright, centered and stable:

$$r = -5\theta^2 - 0.1\dot{\theta}^2 - 0.5p^2 - 0.001\dot{p}^2 + 1$$

The +1 bonus incentivized the robot to stay balanced.

2 Task 2: Train a Stabilizing Policy (20 points)

To train a reinforcement learning policy capable of stabilizing Upkie in its wheeled-inverted-pendulum configuration, I used the custom environment implemented in Task 1 together with the Proximal Policy Optimization (PPO) algorithm from `Stable-Baselines3`. I registered the Upkie environments using: `upkie.envs.register()`. For training (`train_pendulum.py`), I constructed a vectorized environment with 16 parallel copies:

```
env = make_vec_env(ENV_ID, n_envs=16, env_kwargs=dict(frequency=50.0), seed=0)
```

Running multiple environments in parallel significantly increased sample throughput and reduced training time. I used the PPO algorithm with a multilayer perceptron (MLP) policy. The key hyperparameters were:

```
model = PPO(
    "MlpPolicy",
    env,
    learning_rate=3e-4,
    n_steps=1024,
    batch_size=64,
    gamma=0.99,
    gae_lambda=0.95,
    clip_range=0.2,
    ent_coef=0.0,
    n_epochs=10,
    device="auto",
    verbose=1,)
```

To avoid losing progress, I periodically saved intermediate models using:

```
checkpoint_callback = CheckpointCallback(
    save_freq=10_000,
    save_path="./models/pendulum_best/",
    name_prefix="best_model",)
```

I trained the agent for 500,000 simulation timesteps.

I evaluated the trained policy using the separate rollout script (`rollout_policy.py`). The trained model was loaded and run in deterministic mode to measure episode length and cumulative return. A successful policy is one that maintains balance for the full episode horizon (300 steps) without triggering the fall condition. Rollout testing confirmed visually (see video of Task 2 attached) that the trained controller keeps the Upkie upright and stable throughout the episode, demonstrating effective inverted-pendulum stabilization.

3 Task 3: Train a Stabilizing Policy on the Full Model (20 points, BONUS)

In this task, I trained a reinforcement learning policy to stabilize the Upkie using the full six-motor servo model. Unlike the simplified pendulum model in Task 1, the `Upkie-Spine-Servos` environment provides direct moteus-style control over all six joints: left/right hip, left/right knee, and left/right wheel. Each joint exposes position, velocity, torque, PID gain scales, and torque limits at every timestep, making this task substantially more challenging. I first registered the Upkie environments: `upkie.envs.register()`. The training environment was built through a stack of wrappers that define the action interface, simplify observations, and shape the reward:

1. **TorqueActionWrapper**: Replaces the native dictionary action space with a continuous vector in $[-1, 1]^N$, where N is the number of servos. Each component is linearly mapped to a joint's maximum allowable torque. Position and velocity loops are disabled, giving the agent direct torque control: `env = TorqueActionWrapper(env)`.
2. **ServoObsFlattenWrapper**: Converts the per-joint dictionary observations into a flat vector containing positions and velocities. This provides a consistent and MLP-friendly observation space.
3. **StabilizeRewardWrapper**: Implements a shaped reward encouraging upright posture, small torques, and longer survival. A Gaussian centered at pitch = 0 rewards upright balance, while torque penalties promote smooth movements. Large penalties are applied when the robot falls (base pitch exceeding 0.9 rad):

```

reward += exp(-(pitch**2)/0.3)      # upright
reward -= 0.001 * torque_sum        # smoothness
reward += 0.01                      # survival bonus
if abs(pitch) > 0.9:
    terminated = True
    reward -= 10

```

4. **TimeLimit**: Episodes are capped at 300 simulation steps.

The complete wrapped environment is created by: `env = make_training_env()`. To accelerate data collection, I trained using 16 parallel environments: `env = make_vec_env(make_training_env, n_envs=16)`. This efficiently utilizes the available CPU cores and increases the number of timesteps gathered per PPO update. The model was trained for 1.5 million timesteps. I used the PPO algorithm with hyperparameters tuned for torque-control tasks:

```

model = PPO(
    policy="MlpPolicy",
    env=env,
    device="cpu",
    learning_rate=3e-4,
    n_steps=2048,
    batch_size=512,
    n_epochs=10,
    gamma=0.99,
    gae_lambda=0.95,
    clip_range=0.2,
    ent_coef=0.0,
    target_kl=0.02,
    verbose=1,)

```

The trained policy demonstrated partial stabilization of the robot: the agent learned to reduce torques, respond to pitch deviations, and maintain balance for significantly longer than a random policy. Although direct torque control remains a challenging problem and the resulting behavior is less stable than the pendulum case, training clearly improved performance over time. The qualitative rollouts confirmed that the robot was able to maintain upright posture for extended durations before falling (see video of Task 3 attached).